

```

import streamlit as st
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch

# -----
# Page Config
# -----
st.set_page_config(
    page_title="Fake News Detector",
    page_icon="📰",
    layout="centered"
)

# -----
# Custom CSS
# -----
st.markdown("""
<style>
    .stApp {
        background: linear-gradient(135deg, #0f0c29, #302b63, #24243e);
    }

    .main .block-container {
        padding-top: 2.5rem;
        max-width: 760px;
    }

    .hero-title {
        font-size: 2.6rem;
        font-weight: 800;
        text-align: center;
        background: linear-gradient(90deg, #00f5d4, #00bbf9);
        -webkit-background-clip: text;
        -webkit-text-fill-color: transparent;
        margin-bottom: 0.2rem;
    }

    .hero-subtitle {
        text-align: center;
        color: #c9c9e0;
        font-size: 1.05rem;
        margin-bottom: 1.8rem;
    }

```

```

body, .stApp {
    perspective: 1200px;
}

@keyframes floatCard {
    0% { transform: translateY(0px) rotateX(0deg); }
    50% { transform: translateY(-10px) rotateX(1deg); }
    100% { transform: translateY(0px) rotateX(0deg); }
}

.glass-card {
    background: rgba(255, 255, 255, 0.06);
    backdrop-filter: blur(10px);
    border: 1px solid rgba(255, 255, 255, 0.15);
    border-radius: 18px;
    padding: 1.6rem 1.6rem 0.8rem 1.6rem;
    margin-bottom: 1.5rem;
    box-shadow:
        0 25px 50px rgba(0,0,0,0.45),
        0 8px 20px rgba(0, 245, 212, 0.08),
        inset 0 1px 0 rgba(255,255,255,0.15);
    transform-style: preserve-3d;
    animation: floatCard 5s ease-in-out infinite;
    transition: transform 0.15s ease-out, box-shadow 0.3s ease;
    will-change: transform;
}

.glass-card:hover {
    box-shadow:
        0 35px 60px rgba(0,0,0,0.5),
        0 10px 30px rgba(0, 245, 212, 0.25),
        inset 0 1px 0 rgba(255,255,255,0.2);
}

label, .stTextInput label, .stTextArea label {
    color: #e8e8f5 !important;
    font-weight: 600 !important;
    font-size: 0.95rem !important;
}

.stTextInput input, .stTextArea textarea {
    background-color: rgba(255,255,255,0.07) !important;
    color: #ffffff !important;
    border: 1px solid rgba(255,255,255,0.18) !important;
    border-radius: 10px !important;
}

```

```

.stTextInput input::placeholder, .stTextArea textarea::placeholder {
  color: #9a9ab5 !important;
}

div.stButton > button {
  background: linear-gradient(90deg, #00f5d4, #00bbf9);
  color: #0f0c29;
  font-weight: 700;
  border: none;
  border-radius: 10px;
  padding: 0.7rem 1.2rem;
  box-shadow:
    0 8px 0 #008f7a,
    0 14px 20px rgba(0, 245, 212, 0.35);
  transform: translateY(0px);
  transition: transform 0.12s ease, box-shadow 0.12s ease;
}

div.stButton > button:hover {
  transform: translateY(-3px);
  box-shadow:
    0 11px 0 #008f7a,
    0 20px 28px rgba(0, 245, 212, 0.5);
  color: #0f0c29;
}

div.stButton > button:active {
  transform: translateY(5px);
  box-shadow:
    0 3px 0 #008f7a,
    0 6px 10px rgba(0, 245, 212, 0.3);
}

@keyframes popIn3D {
  0% { transform: scale(0.85) rotateX(20deg) translateY(20px); opacity: 0 }
  60% { transform: scale(1.03) rotateX(-3deg) translateY(-4px); opacity: 1 }
  100% { transform: scale(1) rotateX(0deg) translateY(0px); opacity: 1; }
}

.result-real {
  background: linear-gradient(135deg, rgba(0,200,120,0.2), rgba(0,200,120,0
  border: 1px solid rgba(0,200,120,0.5);
  border-radius: 14px;
  padding: 1.1rem 1.4rem;
  text-align: center;
  font-size: 1.4rem;
  font-weight: 800;

```

```

        color: #4ade80;
        margin-bottom: 0.8rem;
        box-shadow: 0 20px 40px rgba(0,200,120,0.25), inset 0 1px 0 rgba(255,255,
        animation: popIn3D 0.5s ease-out;
    }

    .result-fake {
        background: linear-gradient(135deg, rgba(255,60,80,0.2), rgba(255,60,80,0
        border: 1px solid rgba(255,60,80,0.5);
        border-radius: 14px;
        padding: 1.1rem 1.4rem;
        text-align: center;
        font-size: 1.4rem;
        font-weight: 800;
        color: #ff6b81;
        margin-bottom: 0.8rem;
        box-shadow: 0 20px 40px rgba(255,60,80,0.25), inset 0 1px 0 rgba(255,255,
        animation: popIn3D 0.5s ease-out;
    }

    .confidence-text {
        text-align: center;
        color: #d8d8ee;
        font-size: 0.95rem;
        margin-bottom: 0.6rem;
    }

    footer {visibility: hidden;}
    .stCaption, .css-1629p8f { color: #9a9ab5 !important; }
    hr { border-color: rgba(255,255,255,0.1) !important; }
</style>
"""', unsafe_allow_html=True)

# -----
# Mouse-follow 3D Tilt Effect (JS injected into parent page)
# -----
import streamlit.components.v1 as components

components.html("""
<script>
const doc = window.parent.document;

function applyTilt() {
    const card = doc.querySelector('.glass-card');
    if (!card) return;

    card.addEventListener('mousemove', (e) => {

```

```

        const rect = card.getBoundingClientRect();
        const x = e.clientX - rect.left;
        const y = e.clientY - rect.top;
        const centerX = rect.width / 2;
        const centerY = rect.height / 2;
        const rotateX = ((y - centerY) / centerY) * -6;
        const rotateY = ((x - centerX) / centerX) * 6;
        card.style.transform = `rotateX(${rotateX}deg) rotateY(${rotateY}deg) sca
        card.style.animation = 'none';
    });

    card.addEventListener('mouseleave', () => {
        card.style.transform = 'rotateX(0deg) rotateY(0deg) scale(1)';
        card.style.animation = 'floatCard 5s ease-in-out infinite';
    });
}

// Retry until the card exists (Streamlit renders async)
const interval = setInterval(() => {
    const card = doc.querySelector('.glass-card');
    if (card && !card.dataset.tiltApplied) {
        card.dataset.tiltApplied = "true";
        applyTilt();
    }
}, 500);
</script>
""", height=0)

# -----
# Load Model (cached so it loads only once)
# -----
@st.cache_resource
def load_model():
    model_name = "hamzab/roberta-fake-news-classification"
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForSequenceClassification.from_pretrained(model_name)
    model.eval()
    return tokenizer, model

tokenizer, model = load_model()

# -----
# Prediction Function
# -----
def predict_news(title, text):
    # Model expects input in this format: "<title> <content>"
    input_str = f"<title>{title}<content>{text}<end>"

```

```

input_ids = tokenizer.encode_plus(
    input_str,
    max_length=512,
    padding="max_length",
    truncation=True,
    return_tensors="pt"
)

with torch.no_grad():
    output = model(
        input_ids["input_ids"],
        attention_mask=input_ids["attention_mask"]
    )

probs = torch.nn.functional.softmax(output.logits, dim=1)
confidence, predicted_class = torch.max(probs, dim=1)

# Label mapping for this model: 0 = Fake, 1 = Real
labels = {0: "FAKE", 1: "REAL"}
label = labels[predicted_class.item()]
confidence = confidence.item() * 100

return label, confidence, probs[0].tolist()

# -----
# UI
# -----
st.markdown('<div class="hero-title"> Fake News Detector</div>', unsafe_allow_h
st.markdown('<div class="hero-subtitle">AI se check karo ki news REAL hai ya FAKE

st.markdown('<div class="glass-card">', unsafe_allow_html=True)
title_input = st.text_input("📌 News Headline / Title", placeholder="e.g. Scienti
text_input = st.text_area("📝 News Content / Article Text", height=180, placehold
predict_btn = st.button("🔍 Check News", use_container_width=True)
st.markdown('</div>', unsafe_allow_html=True)

if predict_btn:
    if not title_input.strip() and not text_input.strip():
        st.warning("⚠️ Pehle headline ya news text daalo.")
    else:
        with st.spinner("Analyzing news... ⌚"):
            label, confidence, probs = predict_news(title_input, text_input)

        if label == "REAL":
            st.markdown(f'<div class="result-real">✅ REAL NEWS</div>', unsafe_al
        else:
            st.markdown(f'<div class="result-fake">❌ FAKE NEWS</div>', unsafe_al

```

```
st.markdown(f'<div class="confidence-text">Confidence: <b>{confidence:.2f}
st.progress(confidence / 100)

with st.expander("🔍 Detailed Probability Breakdown"):
    st.write(f"Fake Probability: {probs[0]*100:.2f}%")
    st.write(f"Real Probability: {probs[1]*100:.2f}%")

st.markdown("---")
st.caption("Model: hamzab/roberta-fake-news-classification (Hugging Face) | Built
```